NCAOS-Urielle EGS

External Governance Shell · Phase 01 Architecture Report

Phase status	**COMPLETE**	Tests passing	**47 / 47**	Activities done	**36 / 36**	TS source files	**8**
Lines of code	**~1,100**	ADRs	**2**	Detection layers	**4**	Policy profiles	**3**

L1 · External environment

Untrusted inputs — all traffic enters through the shell. Nothing bypasses it.

Ingress gate	Egress filter	Credential boundary

L2 · External governance shell (EGS)

Decoupled observe → judge → enforce loop. Treats core AI as untrusted black box. Fails safe by default.

Observer	Judge	Enforcer	Policy loader	Fail-safe path
I/O boundary tap interface	Premise + authority consistency check	Verdict → PASS / DOWNGRADE / BLOCK action	Zod-validated JSON · 3 builtin profiles	ADR-002: shell error → BLOCK unconditionally

L3 · Detection & scoring · @ncaos/core · 47 tests passing

Four independent containment layers, each producing a typed GovEvent with severity, scope, MTTD, and routing hint.

External gate	Premise consistency	Authority boundary	Continuity watch
detectGate()	detectPremise()	detectAuthority()	detectContinuity()
Flag classification → severity + scope Routes to SOC	Unvalidated claim ratio → severity Routes to Arch review	Override attempt → HIGH / sys scope Routes to Gov review	Equivalence test fail rate → severity; always sys scope Routes to Change mgmt
5 tests ✓	**4 tests ✓**	**3 tests ✓**	**4 tests ✓**

`Scoring functions: computeIntegrity() · computeAuthority() · computeContainmentMetrics() · scoreToContinuityState()`

L4 · Verdict engine · enforce() + failSafeVerdict()

GovEvent x PolicyProfile → deterministic Verdict. No internal state access. govLevel x severity → action.

PASS	DOWNGRADE	BLOCK	FAIL_SAFE
Allow output authority: decision-ready	Downgrade to reference-only Low severity, ENTERPRISE_STRICT	Block output, authority: invalid Med/High under ENTERPRISE_STRICT	Unconditional block on shell error Cannot be disabled by policy

Profiles: ENTERPRISE_STRICT (blocks med+high) · CRITICAL_STRICT (blocks all) · PILOT_OBS (observe only, downgrades high)

L5 · Protected core (black box)

AI model weights, OS kernel logic, internal state. EGS observes I/O only — never modifies or exposes core.

Model runtime	Internal state	OS layer	**EGS never modifies this layer**

L6 · Persistence & audit · structurally isolated from core
Immutable event log, evidence handles (EVD-{ts}-{hash}), policy profiles, tenant configs.

Event store	**Audit log**	**Policy DB**	**Evidence archive**

Core type contracts · @ncaos/core/src/types/contracts.ts

`TenantContext`	PARTNER_ID · INSTANCE_ID · deployment tier — scopes all governance state
`GovEvent`	Primary unit of work: requestId, evidenceHandle, layer, scope, severity, mttdMs, tenant, integrity, authority, routing
`GovState`	Live WebSocket payload: rawExposurePct, governedOutputPct, coreIsolationPct, contextDriftPressurePct, operationalMode
`PolicyProfile`	govLevel x integrityThresholds x continuityThresholds x mttdSlaMs — Zod-validated at runtime
`Verdict`	PASS / DOWNGRADE / BLOCK / FAIL_SAFE — deterministic given event + policy; no internal state access
`OutputAuthority`	decision-ready / reference-only / invalid — structurally granted by shell, not self-reported
`IntegrityState`	Available / Limited / Degraded / Critical with numeric score 0–100; no root-cause inference
`RoutingDecision`	4 routing targets: SOC · Arch review · Gov review · Change mgmt — structural label only

---

Architecture decision records

`ADR-001`	**Type contracts as architectural boundary**	Zod-first schema; all packages import from @ncaos/core. Breaking changes require core version bump; new fields must be optional.
`ADR-002`	**Fail-safe default-deny invariant**	Shell fails BLOCK/invalid on any error — unconditional. Process isolation requirement noted for Phase 2. Chaos test scheduled Phase 6.

Test results summary · 47 / 47 passing

File	Suite	Tests	Status
scoring.test.ts	computeIntegrity	5	PASS
scoring.test.ts	computeAuthority	4	PASS
scoring.test.ts	computeContainmentMetrics	3	PASS
scoring.test.ts	scoreToContinuityState	4	PASS
detection.test.ts	detectGate	5	PASS

detection.test.ts	detectPremise	4	PASS
detection.test.ts	detectAuthority	3	PASS
detection.test.ts	detectContinuity	4	PASS
verdict.test.ts	enforce (ENTERPRISE_STRICT)	6	PASS
verdict.test.ts	enforce (CRITICAL_STRICT)	1	PASS
verdict.test.ts	enforce (PILOT)	2	PASS
verdict.test.ts	failSafeVerdict	1	PASS
verdict.test.ts	validatePolicyConsistency + loadPolicy	5	PASS
	TOTAL	**47**	**ALL PASS ✓**

Key design invariants

EGS never modifies core model weights	The shell observes I/O only. Core is a black box.
Shell fails SAFE (deny) by default	ADR-002: any shell error → FAIL_SAFE verdict unconditionally. Cannot be overridden by policy config.
No root-cause inference	EGS emits impact signals only. No internal AI logic is exposed or inferred.
Evidence trail structurally independent of AI logs	Audit log is isolated from core. EVD handles use FNV hash; Phase 6 upgrades to HMAC-signed entries.
Tenant isolation enforced at type level	TenantContext scopes all governance state. PARTNER_ID isolation is an architectural invariant, not a config option.

Next: Phase 02 — Core shell engine (Week 2–4)

· Implement Observer module: I/O boundary tap interface (process-isolated)

· Implement Judge module: premise consistency + authority checks

· Implement Enforcer module: verdict → containment action mapping

· Build Policy Profile loader (JSON schema + runtime validator)

· Implement FAIL_SAFE default-deny fallback path

· Implement scoring formula: integrity score + continuity score

· Write STRIDE threat model for each shell layer

Deliverable: Core EGS engine — observe → judge → enforce loop with policy-driven verdicts.